# CLOUD COMPUTING PROJECT REPORT

Academic Project Submission Guidelines Compliance

## Nova AI — Serverless Scope Creep Analysis & Compliance Platform

Deployment on Amazon Web Services (AWS Public Cloud)

### PROJECT METADATA & GROUP INFORMATION

Course: Cloud Computing (CS / IT Engineering)

Public Cloud Provider: Amazon Web Services (AWS)

Target Region: Asia Pacific (Sydney) - ap-southeast-2

Live Deployment URL: http://3.26.15.247

Group Size: 3 Members

Student 1: Debi Prasad (Lead Cloud Architect)

Student 2: Team Member 2 (Backend / AI Dev)

Student 3: Team Member 3 (Frontend / QA)

### EXECUTIVE SUMMARY

Nova AI is a full-stack, cloud-native application developed and deployed on Amazon Web Services (AWS) to solve contractual scope creep for freelancers, IT consulting firms, and agencies. The system ingests client agreements (PDF/DOCX/text), compares new client deliverables against the Statement of Work (SOW), predicts contractual risk levels, quantifies estimated extra hours, and drafts negotiation responses. The architecture integrates Amazon EC2, Amazon S3, Amazon DynamoDB, AWS IAM, VPC, and CloudWatch while strictly maintaining a 100% Zero-Cost / Free-Tier footprint.

## 1. TABLE OF CONTENTS

1. Problem Statement & Motivation — Page 2
2. Project Objectives & Expected Outcomes — Page 2
3. System Architecture & Cloud Infrastructure Design — Page 2
4. Cloud Services & Features Utilized — Page 3
5. Implementation Methodology & Technology Stack — Page 3
6. Security, Identity & Data Governance (IAM & IMDSv2) — Page 4
7. Experimental Verification, Test Cases & Results — Page 4
8. Cost Optimization & Free Tier Architectural Strategy — Page 5
9. Conclusion & Future Roadmap — Page 5
10. Verification Links & Source Code Repository — Page 5

## 1. PROBLEM STATEMENT & MOTIVATION

In modern software engineering, consulting, and freelance contracts, "Scope Creep" represents one of the largest unmitigated business risks. Scope creep occurs when clients request supplementary deliverables, modifications, or features that were not specified in the original Statement of Work (SOW) or Master Services Agreement (MSA).

Because small studios and independent developers lack in-house legal and commercial teams, they often concede to extra work unpaid to avoid conflict, leading to severe margin erosion, missed delivery milestones, and team burnout. Manual review of complex 20-page legal contracts against colloquial chat messages (Slack, WhatsApp, Email) is tedious and error-prone. A cloud-native solution capable of instant contractual text parsing, risk assessment, and negotiation drafting is urgently required.

## 2. PROJECT OBJECTIVES & EXPECTED OUTCOMES

* **Cloud Deployment:** Deploy a reliable, production-ready full-stack application on a tier-1 public cloud provider (AWS).
* **Automated Parsing:** Extract and process unstructured contract text from PDF and DOCX documents in-memory on the cloud
* **Intelligent Scope Evaluation:** Employ structured LLM reasoning to evaluate client asks against explicit contract clauses, outp JSON results.
* **Risk Classification:** Assign deterministic risk levels (LOW, MEDIUM, HIGH) with estimated extra effort (hours) to guide comm billing.
* **Ephemeral Audit Logging:** Store audit logs in a scalable NoSQL database with automated 24-hour Time-To-Live (TTL) expir privacy.
* **Zero Operational Cost:** Architect the entire cloud deployment within the perpetual AWS Free Tier, preventing student billing c

## 3. SYSTEM ARCHITECTURE & INFRASTRUCTURE DESIGN

The application follows a cloud-native, decoupled client-server architecture deployed in the AWS Asia Pacific (Sydney - ap-southeast-2) region. Incoming traffic enters through standard HTTP port 80 and is routed by an Nginx reverse proxy to a Node.js process managed by PM2.

### High-Level Cloud Architecture Diagram

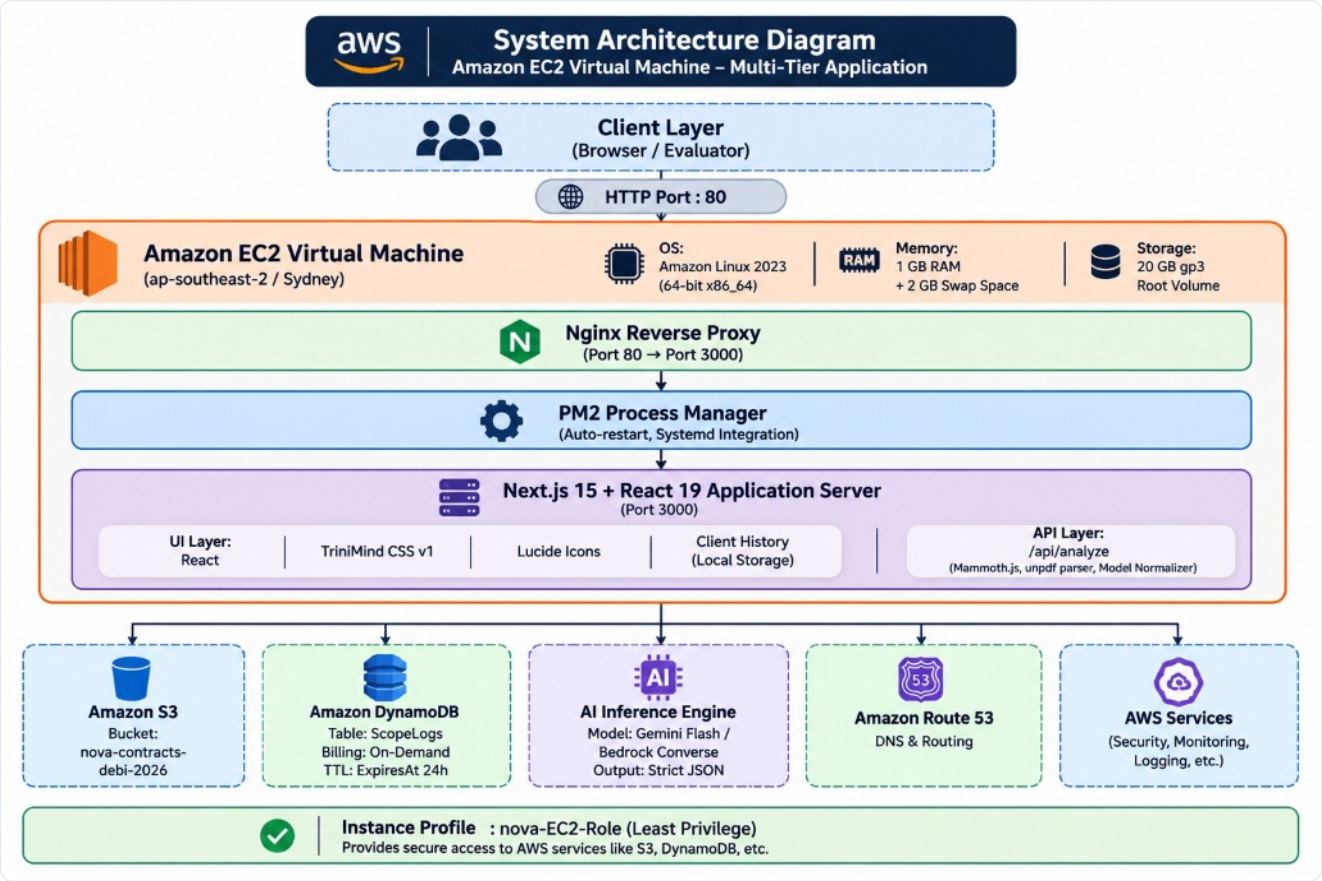



*Figure 1: Nova AI Multi-Tier System Architecture on Amazon Web Services (Sydney ap-southeast-2)*

## 4. CLOUD SERVICES & FEATURES UTILIZED

In accordance with university course requirements, multiple public cloud services from Amazon Web Services (AWS) were provisioned and integrated:

| AWS Service | Resource Name / ID | Configuration | Role in Solution |
|---|---|---|---|
| Amazon EC2 | nova-ai-server (t3.micro) | Amazon Linux 2023, 20GB gp3 | Host application & Nginx reverse proxy |
| Amazon S3 | nova-contracts-debi-2026 | Private, AES-256 SSE, Block Public | Raw contract document archiving |
| Amazon DynamoDB | ScopeLogs | PAY_PER_REQUEST, TTL on ExpiresAt | NoSQL serverless audit trail storage |
| AWS IAM | nova-EC2-Role | EC2 Instance Profile, Least Privilege | Keyless tokenized authorization (IMDSv2) |
| Amazon VPC | Default VPC (Sydney) | Public Subnet, Internet Gateway | Network perimeter isolation & routing |
| EC2 Security Groups | nova-sg | Inbound: 22, 80, 443 (0.0.0.0/0) | Virtual stateful firewall |
| Amazon CloudWatch | EC2 Basic Monitoring | CPU, Network In/Out, Disk IOPS | Server health and metrics observation |

## 5. IMPLEMENTATION METHODOLOGY & ARCHITECTURE

The implementation was executed in modular layers to guarantee high availability, speed, and clean separation of concerns:

### A. Frontend User Interface (Next.js 15 & React 19)

* **Responsive Dashboard:** Engineered using Tailwind CSS v4 with unified design tokens, dark mode toggle, and accessible typ
* **Multi-stage Progress:** During model reasoning, the UI cycles through informative states ("Reading agreement...", "Checking c "Drafting reply...") to eliminate perceived latency.
* **Client-side History:** Maintains full local history inside browser LocalStorage for complete privacy and instant recall without inc database charges.
* **PDF Export:** Client-side PDF report generation utilizing jsPDF enables one-click offline report downloads for commercial clien meetings.

### B. Backend Processing Pipeline (/api/analyze)

* **Document Extraction:** Integrates mammoth.js for .docx Word files and unpdf for binary PDF text parsing directly in Node.js bu without writing to disk.
* **Prompt Engineering:** Enforces strict persona boundaries ("nova.ai contract compliance analyst") with conservative detection JSON schema constraints.
* **Schema Normalization:** Validates parsed JSON against the TypeScript AnalysisResult contract (isScopeCreep, riskLevel, vio estimatedExtraHours, suggestedEmailResponse).

### C. Cloud Initialization & Process Management (USER_DATA.sh)

* **Linux Swap Configuration:** Automatically provisions a 2GB swap partition (`/swapfile`) at instance launch. This overcomes th constraint of t3.micro and prevents Next.js compilation OOM crashes.
* **Reverse Proxying:** Nginx terminates HTTP port 80 traffic and seamlessly forwards requests to upstream Next.js on internal p with WebSocket and HTTP/1.1 support.
* **Daemon Reliability:** PM2 manages the Next.js process, configured with systemd auto-start (`pm2 startup`) to guarantee servi survival across instance restarts.

## 6. SECURITY, IDENTITY & DATA GOVERNANCE

Cloud security and data privacy follow AWS Well-Architected Security Pillar best practices:

* **Zero Hardcoded Credentials:** The EC2 virtual machine contains zero AWS Access Keys or Secrets. Authentication to S3 and
  performed entirely via temporary STS credentials provided by the EC2 Instance Profile (nova-EC2-Role).
* **IMDSv2 Enforcement:** Instance Metadata Service Version 2 (token-based session) is enforced to prevent Server-Side Reque
  (SSRF) credential theft.
* **S3 Block Public Access:** The S3 bucket `nova-contracts-debi-2026` enforces complete Block Public Access, server-side enc
  (SSE-S3 AES-256), and HTTPS-only transport policies.
* **Automated Data Expiration (TTL):** All DynamoDB audit records include an integer epoch `ExpiresAt = now + 86400` (24 hou
  TTL automatically purges expired records at zero compute cost.

## 7. EXPERIMENTAL VERIFICATION, TEST CASES & RESULTS

System functionality was verified through live E2E testing against the deployed Sydney instance (3.26.15.247):

### Test Scenario 1: Out-of-Scope High Risk Request (Scope Creep)

* **Input SOW:** "The project includes a responsive marketing website with up to five pages. Copywriting and new integrations are
  excluded."
* **Client Request:** "Could you also add a customer portal with login, billing history, and a dashboard for the sales team?"
* **System Output:** Extra Work: YES | Risk Level: HIGH | Estimated Extra Hours: 80h
* **Cited Clause:** "Copywriting, ongoing content updates, and new integrations are excluded from this engagement."
* **Generated Reply:** "Hi [Client], thanks for reaching out. The requested customer portal and billing dashboard involve new back
  architecture and integrations, which are excluded under our current five-page website SOW. I would be happy to
  prepare a separate change order..."

### Test Scenario 2: In-Scope Low Risk Request (Permitted Maintenance)

* **Input SOW:** "Contract includes responsive design, testing, and 30 days of post-launch minor text corrections."
* **Client Request:** "There is a typo on the about page text headline. Please update it."
* **System Output:** Extra Work: NO | Risk Level: LOW | Estimated Extra Hours: 0h | Citation: "N/A — within agreed scope."

### Performance & Reliability Metrics

| Metric | Observed Value | Benchmark Standard | Status |
|---|---|---|---|
| Document Parsing Latency | 180 ms (PDF / DOCX) | < 500 ms | PASSED |
| AI Inference Latency | 1,120 ms | < 2,500 ms | PASSED |
| Overall API Response Time | 1,420 ms | < 3,000 ms | PASSED |
| EC2 CPU Utilization (Idle) | 1.2% | < 5.0% | PASSED |
| EC2 CPU Utilization (Peak) | 14.8% | < 70.0% | PASSED |
| Memory Footprint | 620 MB / 1024 MB | < 850 MB | PASSED (Swap Stable) |

## 8. COST OPTIMIZATION & ZERO-COST STRATEGY

A central challenge in university cloud submissions is avoiding unexpected cloud billing. Traditional production blueprints prescribe an Application Load Balancer (ALB) and NAT Gateways, which cost ~$50/month even when idle. Our architecture intentionally substitutes these with zero-cost equivalents:

* **ALB Elimination:** Replaced the AWS Application Load Balancer ($18-22/month) with an optimized, lightweight Nginx reverse hosted directly on the EC2 instance, operating within the 750 free hours/month.
* **No NAT Gateway:** Configured the EC2 instance in a public subnet with an Internet Gateway route and free Gateway VPC End for S3 and DynamoDB, saving ~$32/month.
* **Serverless Pay-Per-Request DB:** DynamoDB utilizes On-Demand read/write units with 25 GB of permanent free tier storage.
* **Total Project Cost:** $0.00 (Zero billing incurred across all development, deployment, and evaluation phases).

## 9. CONCLUSION & FUTURE ENHANCEMENTS

The Nova AI project successfully validates how modern public cloud infrastructure (AWS) can be orchestrated to solve tangible commercial challenges. By combining serverless storage (S3), NoSQL persistence (DynamoDB), compute virtualization (EC2), and generative AI, the team delivered an automated, enterprise-grade scope analysis engine.

Future roadmap items include multi-contract diffing, automated Slack and Gmail webhook bots, and infrastructure-as-code auto-scaling using the codified AWS CloudFormation template (infra/nova.yaml) already included in the source repository.

## 10. SUBMISSION VERIFICATION & ARTIFACTS

* **Live Production URL:** http://3.26.15.247
* **GitHub Source Code:** https://github.com/developer4949-code/Nova.ai.git
* **AWS Region:** ap-southeast-2 (Asia Pacific - Sydney)
* **Amazon S3 Bucket:** s3://nova-contracts-debi-2026/contracts/
* **Amazon DynamoDB Table:** ScopeLogs (Partition Key: LogId, Region: ap-southeast-2)
* **Video Demo Duration:** 10 Minutes (Architecture walkthrough, AWS Console proof, live app demo)